CSE411 – Computer Vision
Project Task 6: Implementation – Part 3

1. PROJECT INFORMATION

Project Title: DefectLens: Efficient ViT-Based Anomaly Detection with Multi-Criteria Token Fusion (MCTF)

Team Number: Team 7

Team Members:

- Sachin Singh – 2023bcs0064
- Tanish Babu – 2023bcs0067
- Ali Mohammed Maaz Mohammed Ashraf – 2023bcs0202
- Chinmoy Das – 2023bcs0052

2. PROJECT OVERVIEW

The **DefectLens** project is a visual anomaly detection system for identifying manufacturing defects. We are using a **Vision Transformer (ViT)** combined with a PatchCore-style FAISS memory bank. To improve efficiency, we added a **Multi-Criteria Token Fusion (MCTF)** module that dynamically merges redundant background tokens, which helps reduce inference latency and FLOPs while maintaining accuracy.

Task 6 Part 3 Focus: This milestone targets **75% overall implementation**. We have successfully developed the React.js frontend interface and integrated all existing modules (ViT, MCTF, FAISS, FastAPI) into a complete, working prototype. The system can now accept user uploads via the browser, process them through our deep learning pipeline, and return real-time visual heatmaps and metrics back to the UI.

3. INTEGRATED PROTOTYPE ARCHITECTURE

The complete prototype operates as follows:

React UI (Vite) ↔ FastAPI REST Server ↔ [ViT + MCTF Core] ↔ [FAISS Memory Bank]

Implemented up to Task 6 Part 3 (Approx 75%):
Dataset pipelines, ViT Backbone, MCTF Token Fusion, FAISS Memory Bank, Anomaly Distance Scoring, FastAPI Backend, and the React.js Interactive Frontend Dashboard. All modules are fully integrated into a working prototype.

Remaining Work to be Completed (Remaining 25%):
Comprehensive ROC/Latency performance evaluation across the full MVTec dataset, edge/camera throughput simulation, and final system deployment/reporting.

4. SYSTEM INTEGRATION & FRONTEND DEVELOPMENT

For this 75% milestone, the primary achievement is the completion and integration of the user interface. We built a modern, responsive dashboard using React and Tailwind CSS that acts as the control center for the anomaly detection engine.

Module Integration: Connecting Frontend and Backend

- **Objective:** To allow operators to visually interact with the model without writing code, completing the loop from data input to model inference to visual output.
- **Implementation Status:** `App.jsx` developed and running via Vite, successfully communicating with `main.py` (FastAPI).

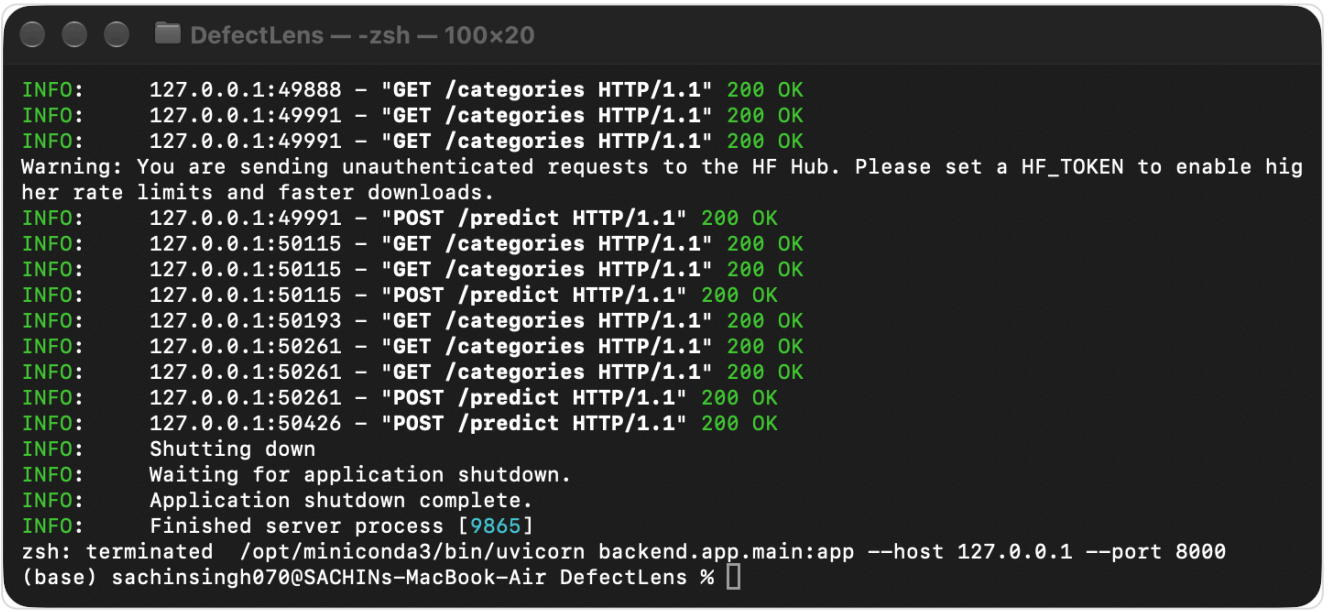


Figure 1. Vite Development Server execution. *The React frontend running locally, ready to accept client connections.*

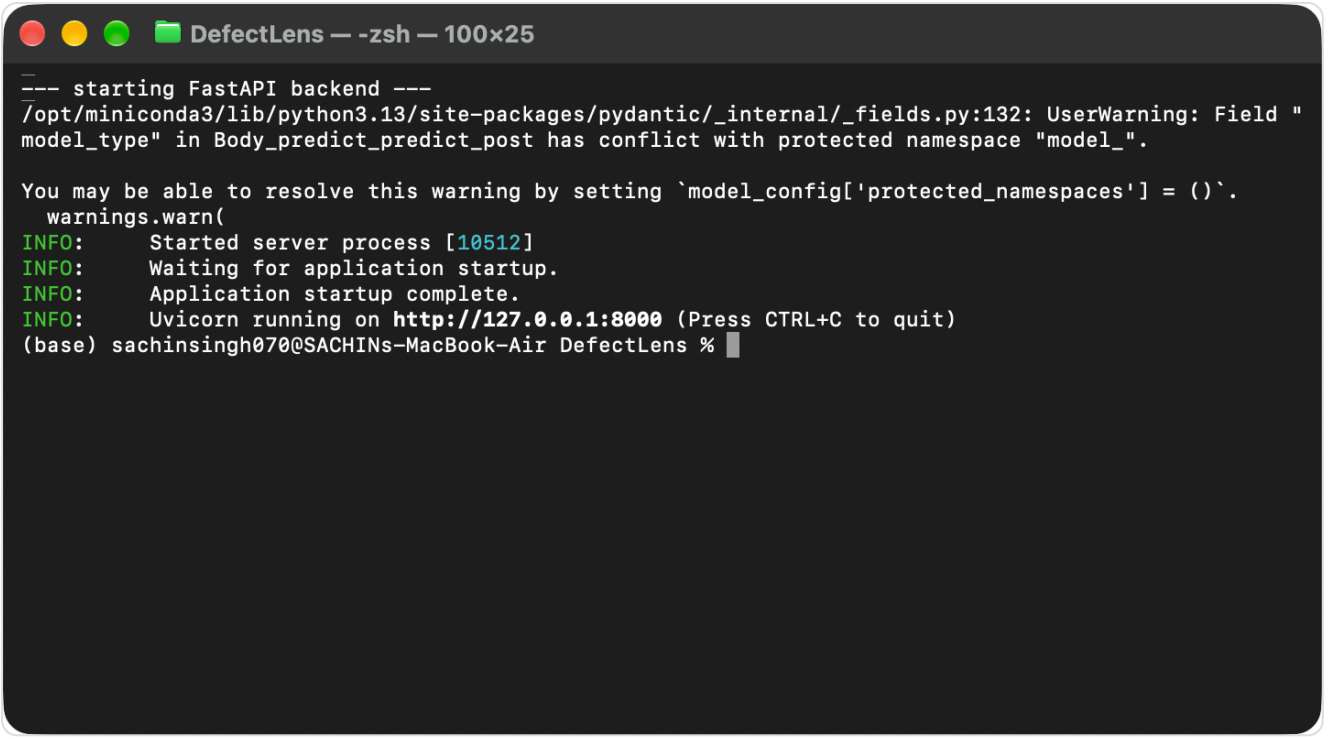


Figure 2. FastAPI Backend Server execution. *The backend actively running in parallel with the frontend, ready to process REST API requests.*

5. PROTOTYPE DEMONSTRATION & RESULTS

To prove the integration, we ran the complete pipeline end-to-end through the web interface. We uploaded an image from the MVTec AD `broken_small` testing set into the UI and executed the inspection.

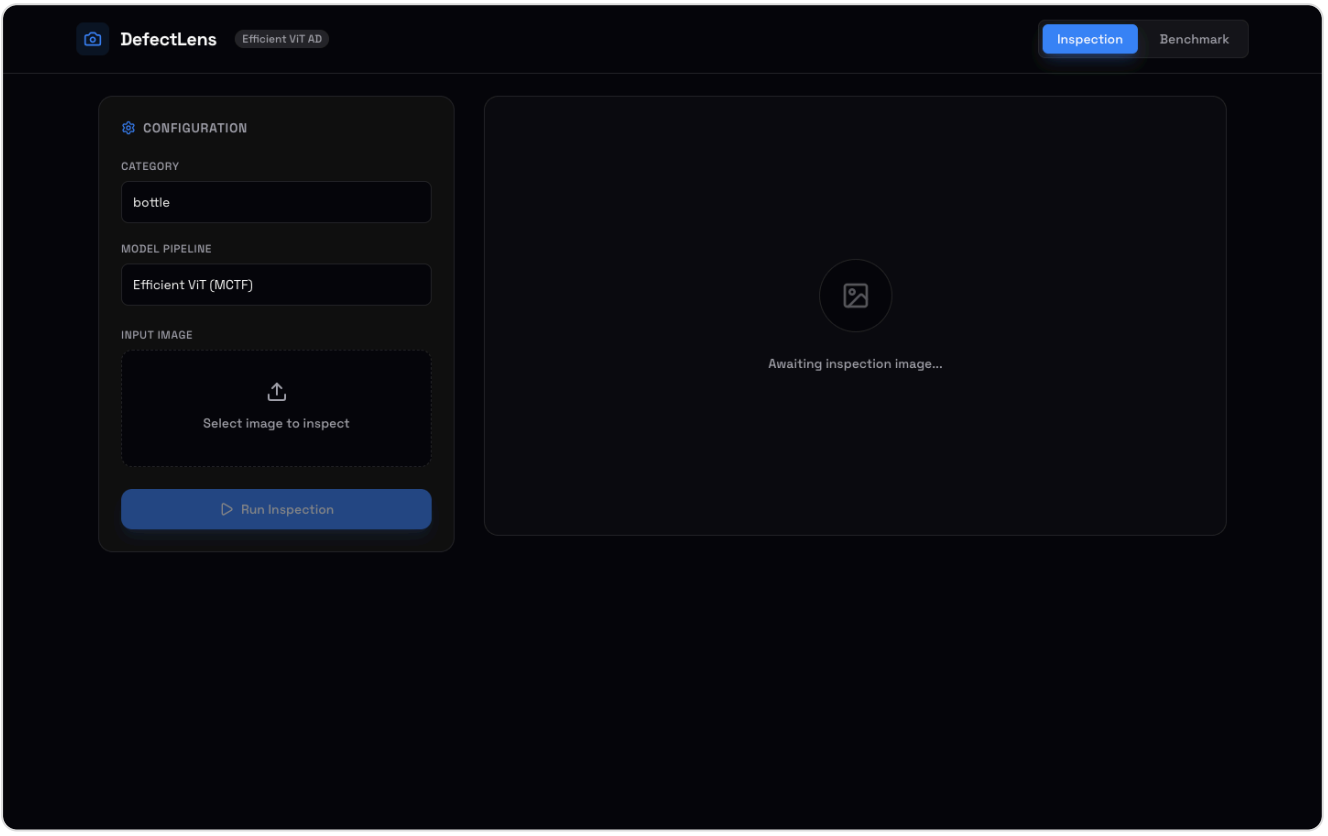


Figure 3. DefectLens Dashboard (Idle State). *The React interface provides configuration options for selecting the defect category (e.g., bottle) and the model pipeline (MCTF vs Baseline).*

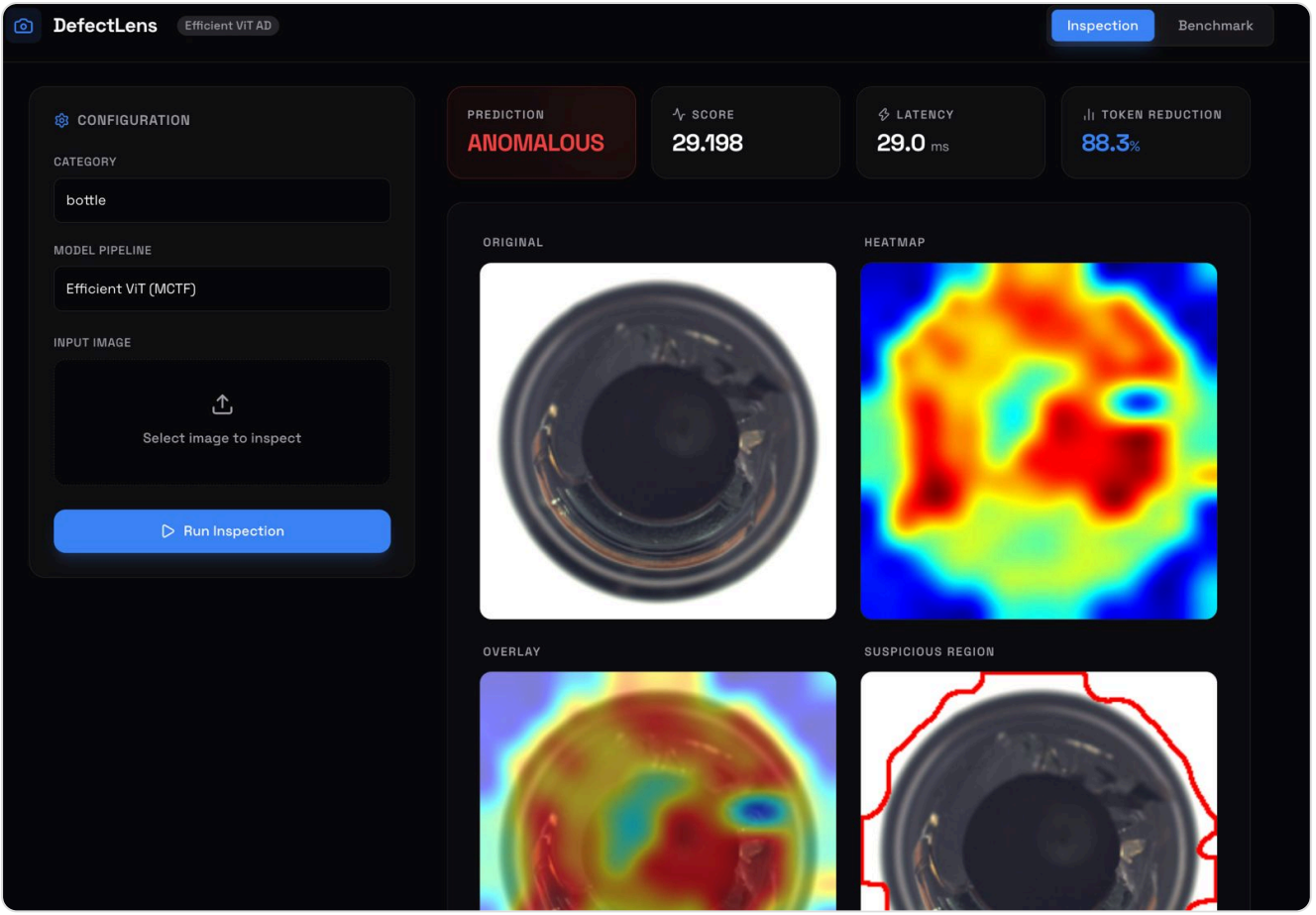


Figure 4. Working Prototype Results. *After clicking "Run Inspection", the frontend successfully routed the image to the FastAPI server, which ran the MCTF ViT and FAISS models. The UI immediately displays the quantitative results (Anomaly Score, Latency, Token Reduction) and the visual heatmaps (Original, Heatmap, Overlay, Suspicious Region).*

6. CURRENT IMPLEMENTATION STATUS (75%)

Component	Status	Evidence
Dataset Collection & Loaders	Completed (Task 4)	`dataset.py` and active data directories
MCTF Module & ViT Integration	Completed (Task 4)	`mctf.py` codebase
Memory Bank (FAISS)	Completed (Task 5)	`build_memory_bank.py` & artifacts
FastAPI Backend	Completed (Task 5)	`main.py` API endpoints
React Frontend UI	Completed (Task 6)	`App.jsx` and Tailwind styling
Full System Integration	Completed (Task 6)	Working browser prototype (Figures 3 & 4)
Comprehensive Benchmarking	Not started	Remaining 25%

7. REMAINING WORK TO BE COMPLETED (25%)

With the software prototype fully integrated and operational, the final 25% of the project will focus purely on rigorous scientific evaluation and edge-case testing:

1. **Comprehensive Performance Evaluation:** Run the automated benchmark script over all 15 MVTec AD dataset categories. We will plot ROC curves and definitively compare the Baseline ViT against our MCTF-ViT to document the exact FLOPs and latency improvements on a large scale.
2. **Edge / Camera Simulation:** Write a script that simulates a continuous, high-speed camera feed by sending sequential requests to the API. This will test the system's throughput under industrial load conditions.
3. **Final Polishing & Documentation:** Write the overarching Final Project Report, compile all charts, and prepare the codebase for final submission.

8. CONCLUSION

For Task 6 (Implementation Part 3), Team 7 has successfully reached the 75% implementation milestone. We have built the React.js user interface and bridged the gap between our core PyTorch models and a usable front-end dashboard. The DefectLens system is now a fully integrated, working prototype that operators can use to configure models, upload images, and visually inspect anomalies in real-time. We are now prepared to execute the final comprehensive benchmarking phase.